

**Федеральное агентство по образованию
Государственное образовательное учреждение высшего образования
Московский политехнический университет**

Кафедра Прикладная информатика

**Дисциплина
Проблемно-ориентированные языки**

**Лекция №1 НАЗНАЧЕНИЕ И ПРИМЕНЕНИЕ JAVASCRIPT,
ОБЩИЕ СВЕДЕНИЯ**

JavaScript — это язык управления сценариями просмотра гипертекстовых страниц Web на стороне клиента.

Основная идея **JavaScript** состоит в возможности изменения значений атрибутов HTML-контейнеров и свойств среды отображения в процессе просмотра HTML-страницы пользователем. При этом перезагрузки страницы не происходит. На практике это выражается в том, что можно, например, изменить цвет фона страницы или интегрированную в документ картинку, открыть новое окно или выдать предупреждение.

Размещение кода JavaScript на HTML-странице

Во-первых, исполняет **JavaScript-код** браузер. В него встроен *интерпретатор JavaScript*. Следовательно, выполнение программы зависит от того, когда и как этот интерпретатор получает управление. Это, в свою очередь, зависит от функционального применения кода.

В общем случае можно выделить четыре способа функционального применения JavaScript:

- гипертекстовая ссылка (схема URL);
- обработчик события (в атрибутах, отвечающих событиям);
- подстановка (entity);
- вставка (контейнер <SCRIPT>).

Способ 1: URL-схема "JavaScript:"

Схема URL (Uniform Resource Locator) — это один из основных элементов Web-технологии. Каждый информационный ресурс в Web имеет свой уникальный URL. URL указывают в атрибуте HREF контейнера A, в атрибуте SRC контейнера IMG, в атрибуте ACTION контейнера FORM и т.п.

Все URL подразделяются на *схемы доступа*, которые зависят от протокола доступа к ресурсу, например, для доступа к FTP-архиву применяется схема ftp, для отправки электронной почты — схема mailto.

Тип схемы определяется по первому компоненту URL, например:
<http://its.kpi.ua/directory/page.html>

В данном случае URL начинается с http — это и есть задание схемы доступа (схема http).

Основной задачей языка программирования гипертекстовой системы является программирование гипертекстовых переходов. Это означает, что при выборе той или иной гипертекстовой ссылки вызывается программа реализации гипертекстового перехода. В Web-технологии стандартной программой, вызываемой при гипертекстовом переходе, является программа загрузки страницы (т.е. при клике по ссылке загружается страница с указанным URL).

JavaScript позволяет поменять стандартную программу на программу пользователя. Для того чтобы отличить стандартный переход по протоколу HTTP от перехода, программируемого на JavaScript, разработчики языка ввели новую схему URL— JavaScript:

```
<A HREF="JavaScript:код_программы">...</A>  
<FORM ACTION="JavaScript:код_программы" ...> ... </FORM>
```

В данном случае текст "код_программы" обозначает программу-обработчик на JavaScript, которая вызывается при выборе гипертекстовой ссылки в первом случае и при отправке данных формы (нажатии кнопки Submit) — во втором.

Например, при нажатии на гипертекстовую ссылку "Клихни здесь" можно получить окно предупреждения:

```
<A HREF="JavaScript:alert('Внимание!!!');">Клихни здесь</A>
```

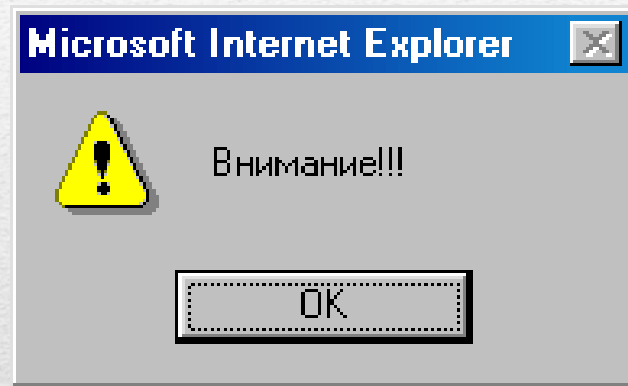


Рисунок 1.1 - Окно предупреждения

В URL можно размещать сложные программы и вызовы функций. Таким образом, при программировании гипертекстового перехода JavaScript-интерпретатор получает управление после того, как пользователь "кликнул" по гипертекстовой ссылке.

Способ 2: обработчики событий

Такие программы, как *обработчики событий*, указываются в атрибутах контейнеров, с которыми эти события связаны. Например, при нажатии на кнопку происходит событие Click и соответственно вызывается обработчик этого события onClick:

```
<FORM>  
<INPUT TYPE=button VALUE="Кнопка"  
      onClick="alert('Вы нажали кнопку');">  
</FORM>
```

А в момент завершения полной загрузки документа (он связан с контейнером <BODY>) происходит событие Load и, соответственно, будет вызван обработчик этого события onLoad:

```
<BODY onLoad="alert('Приветствуем!');">  
...  
</BODY>
```


Способ 3: подстановки

Подстановки (entity) поддерживаются только браузером Netscape Navigator 4.0. Они встречаются на Web-страницах довольно редко. Тем не менее это достаточно мощный инструмент генерации HTML-страницы на стороне браузера. Подстановки имеют формат: &{код_программы}; и используются в качестве значений атрибутов HTML-контейнеров. В следующем примере поле ввода INPUT будет иметь, в качестве значения по умолчанию, адрес текущей страницы, а размер поля будет равным количеству символов в этом адресе.

```
<HTML>
<HEAD>
<SCRIPT>
function l()
{
str = window.location.href;
return(str.length);
}
</SCRIPT>
</HEAD>
<BODY>
<FORM><INPUT TYPE=text SIZE="& {l()};"
VALUE="& {window.location.href};">
</FORM>
</BODY>
</HTML>
```


В случае подстановки JavaScript-интерпретатор получает управление в момент разбора браузером (компонент *парсер*) HTML-документа. Как только парсер встречает конструкцию `&{..}`; у атрибута контейнера, он передает управление JavaScript-интерпретатору, который, в свою очередь, после исполнения кода это управление возвращает парсеру. Таким образом, данная операция аналогична подкачке графики на HTML-страницу.

В Internet Explorer подстановки не поддерживаются, поэтому пользоваться ими следует аккуратно. Прежде чем выдать браузеру страницу с подстановками, нужно проверить тип этого браузера. Альтернативой подстановкам в Internet Explorer можно считать *динамические свойства* стиля. Например, следующий фрагмент создаст поле ввода, ширина которого в пикселях (px) равна количеству символов в адресе страницы, умноженному на 10:

```
<INPUT TYPE=text style="width:expression(10*location.href.length+'px')">
```


Способ 4: вставка (контейнер <SCRIPT>)

Контейнер SCRIPT — это развитие подстановок до возможности генерации текста документа JavaScript-кодом. В этом смысле применение SCRIPT аналогично Server Side Includes, т.е. генерации страниц документов на стороне сервера.

При разборе документа HTML-парсер передает управление JavaScript-интерпретатору после того, как встретит тег начала контейнера <SCRIPT>. Интерпретатор получает на исполнение весь фрагмент кода внутри контейнера SCRIPT и возвращает управление HTML-парсеру для обработки текста страницы после тега конца контейнера </SCRIPT>.

Помещать JavaScript-код на HTML-странице с помощью контейнера `<SCRIPT>` можно двумя способами.

- Первый состоит в написании текста кода непосредственно внутри этого контейнера:

```
<SCRIPT>  
a = 5;  
</SCRIPT>
```

- Второй способ состоит в том, чтобы вынести код JavaScript в отдельный файл, например, `myscript.js` (расширение может быть любым), и затем включить его в HTML-страницу следующим образом:

```
<SCRIPT SRC="myscript.js"></SCRIPT>
```


Этот способ удобен, когда один и тот же скрипт планируется использовать на разных HTML-страницах. Обратите внимание, что при наличии атрибута SRC содержимое контейнера `<SCRIPT>` пусто, и это не случайно: согласно спецификации HTML, если скрипт подключается из внешнего файла, то скрипт, написанный между тэгами `<SCRIPT>` и `</SCRIPT>`, если таковой имеется, будет проигнорирован браузером.

Здесь уместно небольшое замечание, которое позволит Вам избежать одной частой ошибки. Между тэгами `<SCRIPT>` и `</SCRIPT>` не должно встречаться последовательности символов `</SCRIPT>` в любом контексте. Например, следующий пример работать не будет:

```
<SCRIPT>  
alert('</script>');  
</SCRIPT>
```

Дело в том, что специфика разбора HTML-документа браузером такова, что он сначала определяет границы скрипта, а потом уже передает его интерпретатору JavaScript.

Чтобы пример заработал, достаточно, например, написать `alert('<\script>')` (т.к. комбинация `"\"` выводит на экран символ `"/"`), либо разбить строчку на две: `alert('</scr'+ 'ipt>')`.

Контейнер SCRIPT выполняет две основные функции:

- **размещение кода** внутри HTML-документа;
- **условная генерация** HTML-разметки на стороне браузера.

Первая функция аналогична декларированию переменных и функций, которые потом можно будет использовать в качестве программ переходов, обработчиков событий и подстановок. Вторая— это подстановка результатов исполнения JavaScript-кода в момент загрузки или перезагрузки документа.

Размещение кода внутри HTML-документа

Код можно разместить либо в заголовке документа (внутри контейнера HEAD) либо в теле документа (внутри контейнера BODY).

Код в заголовке документа размещается внутри контейнера SCRIPT. В следующем примере мы декларировали функцию `time_scroll()` в заголовке документа, а потом вызвали ее как обработчик события Load в теге начала контейнера BODY.

Функция `time_scroll()` вызывается по окончании полной загрузки документа (обработчиком `onLoad`). Она заносит текущую дату и время (`new Date`) в переменную `d`. Затем записывает текущее время в формате ЧЧ:ММ:СС в `window.status`, тем самым оно будет отображаться в поле статуса окна браузера (подробнее о нем рассказано в лекции 4). Наконец, она откладывает (`setTimeout`) повторный вызов самой себя на 1000 миллисекунд (т.е. 1 секунду). Таким образом, каждую секунду в поле статуса будет отображаться новое время.

```
<HTML>
<HEAD>
<SCRIPT>
function time_scroll()
{var d = new Date();
window.status = d.getHours()
+ ':' + d.getMinutes()
+ ':' + d.getSeconds();
setTimeout('time_scroll()',1000);}
</SCRIPT>
</HEAD>
<BODY onLoad="time_scroll()">
<H1>Часы в строке статуса</H1>
</BODY>
</HTML>
```

Пример 1.2. Часы в поле статуса окна

Комментарии в HTML и JavaScript

В программе JavaScript можно оставлять *комментарии*, которые игнорируются JavaScript-интерпретатором и служат как пояснения для разработчиков.

Однострочные комментарии начинаются с символов //. Текст начиная с этих символов и до конца строки считается комментарием.

Многострочный комментарий заключается между символами /* и */ и может простирается на несколько строк.

```
<SCRIPT>
```

```
a=5; // однострочный комментарий
```

```
/* Многострочный
```

```
комментарий */
```

```
</SCRIPT>
```

Регистр символов

Как Вы, наверное, знаете, язык HTML является регистро-независимым. Вследствие этого, контейнер `<SCRIPT>` можно писать как `<script>`, его атрибуты — как `Type`, `LANGuage` и `src`, значение атрибутов, указывающих язык, — как `"JavaScript"` и `"TEXT/JavaScript"`. Разумеется, значение атрибута `SRC`, т.е. имя файла, следует писать точно так, как файл назван в операционной системе.

Напротив, язык же JavaScript — регистро-зависимый. Это означает, что все переменные, функции, ключевые слова и т.п. должны набираться в том же регистре, в каком они заданы в языке или в программе пользователя.

Например, если Вы объявили переменную `var myText='Привет'`, то в дальнейшем ее можно использовать только как `myText`, но не `MyText`.

Это касается и всех встроенных объектов, свойств и методов языка. Например, объектом является `document`. Вызов `document.write()` нельзя записать как `Document.write()` или `document.Write()`. К свойству объекта `document`, задающему цвет фона Web-страницы, можно обратиться только как `document.bgColor`, а метод этого же объекта, выдающий элемент с заданным идентификатором "id5", можно вызвать только как `document.getElementById("id5")`.